

APP-5258 · Validación biométrica Facephi — Identity Validation V2

Microservicio: `onboarding-micro-person`

Stack: .NET 8 · MediatR · FluentValidation

Endpoint expuesto: `POST /facephi/validate`

Endpoint integrado: `POST /onboarding/v2/identity` (Facephi Identity API)

Épica: APP-5222 · **Bloquea:** APP-5223 — Enrolamiento de Soft Token mediante validación biométrica Facephi

Actualizado: 10 de agosto de 2026

Qué cambió respecto a la primera versión de este documento. El equipo definió que los endpoints no deben tener semántica de Soft Token: son genéricos de Facephi. Se descartaron los cinco endpoints propios, la base de datos MongoDB nueva y la máquina de estados. El alcance real es **un solo endpoint que integra Identity Validation V2**, con los datos mínimos que ese servicio recibe.

Índice

1. Contexto y decisiones cerradas
 2. Contrato de Facephi — Identity Validation V2
 3. Tablas de códigos de resultado
 4. Lo que ya está implementado
 5. Lo que falta: regla de decisión y mapeo a HTTP
 6. Lo que falta: headers, binding y limpieza
 7. Checklist de cierre
 8. Preguntas cerradas y abiertas
-

1. Contexto y decisiones cerradas

Facephi ya estaba integrado en `onboarding-micro-person` para el proceso de onboarding. APP-5258 **no crea un microservicio ni un flujo paralelo**: añade dentro del mismo micro la operación de validación biométrica que consumirá el enrolamiento de Soft Token (APP-5223).

1.1 Decisiones tomadas por el equipo

Decisión	Resultado
¿Endpoints propios de Soft Token?	No. Los endpoints son genéricos de Facephi. Nada de rutas, carpetas ni clases con "softtoken" en el nombre
¿Qué se integra?	POST <code>/onboarding/v2/identity</code> — Identity Validation V2
¿Datos mínimos?	Los que recibe ese endpoint: <code>token1</code> , <code>bestImageToken</code> , <code>method</code> y opcionalmente <code>tracking</code>
¿ requestId ?	No es relevante para nosotros
¿Persistencia en Mongo?	Descartada. La operación es sin estado: entra la captura, sale el veredicto
¿ token2 (dorso del documento)?	No aplica. Identity V2 no tiene ese campo. El modelo original del ticket debe corregirse
¿Identity V2 o Authenticate User V2?	Identity V2 , confirmado: el cliente escanea su cédula nuevamente al activar el token

1.2 Por qué Identity V2 y no Authenticate User V2

`authenticateUser/v2` compara el `bestImageToken` del momento contra un **template biométrico almacenado** (`registeredTemplateRaw`) durante el onboarding. Evita volver a escanear el documento, pero obliga a persistir biometría por usuario.

Identity V2 compara **la foto del documento capturada en el momento** contra el selfie. Como el flujo de Soft Token sí vuelve a pedir la cédula al cliente, Identity V2 es el correcto y **no hay que almacenar nada**. Esta decisión elimina de raíz el requisito de la base de datos nueva.

2. Contrato de Facephi — Identity Validation V2

POST {FACEPHI_IDENTITY_API_BASE_URL}/onboarding/v2/identity

Content-Type: application/json

2.1 Headers

Header	Requerido	Valor
x-api-key	Sí	API key de autorización
family	Sí cuando se envía tracking	OnBoarding

La documentación en español escribe OnBoarding y la inglesa Onboarding . Verificar cuál acepta el ambiente antes de dar por buena la integración.

2.2 Request

Campo	Tipo	Requerido	Descripción
token1	string	Sí	Imagen de referencia para la comparación facial: base64 abierto o TokenFaceImage , según el método
bestImageToken	string	Sí	bestImage tokenizada generada por el widget Selphi
method	string	Sí	Método de comparación: "3" o "5"
tracking	object	No	Información de seguimiento
tracking.extraData	string	No	Token generado por el SDK Mobile/Web
tracking.operationId	string	No	Identificador de operación del SDK (UUID)

Especificación de method

Método	Qué se envía en token1	Requiere
"3"	Imagen abierta en base64 del frente de la cédula, donde está el rostro	—
"5"	Token generado por el recorte de la foto del documento (TokenFaceImage)	Widget SelphID Mobile

Ambos métodos usan bestImageToken como segunda imagen. La app decide cuál usar según cómo capture la cédula.

```
{
  "token1": "/9j/4AAQSkZJRgABAQAASAB...",
  "bestImageToken": "BAMBAQLNHJwGPj...",
  "method": "3",
  "tracking": {
    "extraData": "BQABAQG2gBNjuHN4kLmPqYf7R...",
    "operationId": "123e4567-e89b-12d3-a456-426614174000"
  }
}
```

2.3 Response 200

Campo	Tipo	Descripción
serviceTransactionId	string	Identificador de la transacción en la API
serviceResultCode	integer	Resultado de la ejecución del servicio (no del veredicto biométrico)
serviceResultLog	string	Detalle de la ejecución
serviceTime	string	Tiempo de procesamiento en milisegundos
facialAuthenticationResult	integer	Resultado de la coincidencia facial
facialAuthenticationLog	string	Detalle de la coincidencia facial
facialAuthenticationSimilarity	number	Similitud facial. 1.0 = 100 %
passiveLivenessResult	integer	Resultado de la prueba de vida pasiva
passiveLivenessLog	string	Detalle de la prueba de vida

```
{
  "serviceTransactionId": "2db602ee-3564-4304-af95-92a52eaae12d",
  "serviceResultCode": 0,
  "serviceResultLog": "[identity] Service executed ok",
  "serviceTime": "2235",
  "facialAuthenticationResult": 3,
  "facialAuthenticationLog": "Positive",
  "facialAuthenticationSimilarity": 0.99214232,
  "passiveLivenessResult": 3,
  "passiveLivenessLog": "Live"
}
```

Identity V2 rompe la convención de los demás endpoints de Facephi. `authenticateUser/v2` y `evaluatePassiveLiveness` usan `serviceFacialAuthenticationResult` y `serviceLivenessResult`; Identity V2 usa `facialAuthenticationResult` y `passiveLivenessResult`, sin prefijo. No copiar modelos entre endpoints.

2.4 Errores HTTP

Código	Significado
400	Request inválido — nuestro payload está mal armado
401	API key ausente o inválida
403	La API key no tiene permiso sobre el recurso
502	Facephi devolvió una respuesta inválida
504	Timeout del endpoint

3. Tablas de códigos de resultado

3.1 serviceResultCode

Valor	Descripción	HTTP
0	La ejecución del servicio fue exitosa: el módulo procesó la solicitud correctamente	200

Crítico: `serviceResultCode = 0` significa "el servicio corrió bien", **no** "la persona fue aprobada". En la documentación de `evaluatePassiveLiveness`, un rechazo por prueba de vida devuelve `serviceResultCode: 0` con `serviceResultLog: "NoLive"`. El veredicto está en los otros dos campos.

3.2 facialAuthenticationResult — coincidencia facial

Código	Resultado	Interpretación
0	NONE	No se pudo realizar la verificación → reintentable
1	NEGATIVE	Los rostros no coinciden → rechazo
3	POSITIVE	Los rostros coinciden → único valor aprobado
4	NONE BECAUSE POSE EXCEED	Posición del rostro inválida → reintentable
5	NONE BECAUSE INVALID EXTRACTIONS	No se pudo extraer el patrón facial → reintentable

3.3 `passiveLivenessResult` — prueba de vida

Código	Resultado	Interpretación
3	Live	Sujeto vivo → único valor aprobado
17	NoLive	No se detectó vida → rechazo
1	Spoof	Deprecado, usar NoLive → rechazo
0	None	No se pudo evaluar → reintentable
2	Uncertain	Deprecado → reintentable
4	NoneBecauseBadQuality	Mala calidad de imagen → reintentable
5	NoneBecauseFaceTooClose	Rostro muy cerca de los bordes → reintentable
6	NoneBecauseFaceNotFound	No se detectaron rostros → reintentable
7	NoneBecauseFaceTooSmall	Rostro muy pequeño → reintentable
8	NoneBecauseAngleTooLarge	Ángulo excede el límite → reintentable
9	NoneBecauseImageDataError	Error de formato de imagen → reintentable
10	NoneBecauseInternalError	Error interno de Facephi → error técnico
11	NoneBecauseImagePreprocessError	Error de preprocesamiento → reintentable
12	NoneBecauseTooManyFaces	Demasiados rostros → reintentable
13	NoneBecauseFaceTooCloseToBorder	Rostro muy cerca del borde → reintentable
14	NoneBecauseFaceCropped	Rostro recortado → reintentable
15	NoneBecauseLicenseError	Error de licencia → error técnico
16	NoneBecauseFaceOccluded	Rostro ocluido → reintentable
18	NoneBecauseEyesClosed	Ojos cerrados → reintentable

Por qué la distinción importa

Un cliente con mala luz (4 , 16 , 18) **no es un fraude**: debe poder recapturar. Un 1 en coincidencia facial o un 17 en prueba de vida sí son rechazos que deben cortar el flujo de activación del Soft Token. Devolver lo mismo en ambos casos convierte un problema de iluminación en un cliente bloqueado.

4. Lo que ya está implementado

Verificado contra la documentación oficial, campo por campo. **La integración es correcta.**

Elemento	Doc	Implementación
URL	POST /onboarding/v2/identity	✓
Campos obligatorios del request	token1 , bestImageToken , method	✓
tracking opcional	✓	✓ y se omite del payload cuando no viene
Valores de method	"3" o "5"	✓ validados
operationId como UUID	✓	✓ validado
Campos de respuesta	9 campos	✓ los 9 mapeados
Logs sin datos biométricos	—	✓ sólo método, códigos y transactionId
Ruta genérica, sin "softtoken"	requisito del equipo	✓ /facephi/validate

4.1 Estructura

```
src/micro-person-api/  
├─ Endpoints/FacePhi.cs                → POST /facephi/validate  
├─ Application/Facephi/Commands/ValidateIdentityV2Cmd.cs → Cmd + Validator + Handler  
├─ Common/Models/Biometric/PassiveLivenessResults.cs    → PassiveLivenessResult  
├─ Common/Interfaces/Biometric/IFacePhiService.cs  
├─ Services/Biometric/FacePhiService.cs                → EvaluatePassiveLivenessToken  
tests/UnitTests/Application/SoftTokenFacephi/  
├─ ValidateIdentityV2CmdTests.cs                → validador + handler
```


4.2 Comando y validaciones

```
public class ValidateIdentityV2Cmd : IRequest<PassiveLivenessResult>
{
    public string Token1 { get; set; } = string.Empty;
    public string BestImageToken { get; set; } = string.Empty;
    public string Method { get; set; } = string.Empty;
    public FacephiTrackingExtraData? Tracking { get; set; }
}

public class ValidateIdentityV2CmdValidator : AbstractValidator<ValidateIdentityV2Cmd>
{
    public ValidateIdentityV2CmdValidator()
    {
        RuleFor(x => x.Token1).NotEmpty().WithMessage("token1 is required.");
        RuleFor(x => x.BestImageToken).NotEmpty().WithMessage("bestImageToken is required.");
        RuleFor(x => x.Method)
            .NotEmpty().WithMessage("method is required.")
            .Must(method => method is "3" or "5").WithMessage("method must be 3 or 5.");

        When(x => x.Tracking is not null, () =>
        {
            RuleFor(x => x.Tracking!.OperationId)
                .Must(operationId => string.IsNullOrEmpty(operationId)
                    || Guid.TryParse(operationId, out _))
                .WithMessage("tracking.operationId must be a valid UUID.");
        });
    }
}
```

4.3 Construcción del payload

El servicio omite el objeto `tracking` completo cuando no hay datos, en lugar de enviarlo con nulos:

```
object payload =
    string.IsNullOrEmpty(request.TrackingToken) &&
    string.IsNullOrEmpty(request.OperationId)
? new
{
    token1 = request.Token1,
    bestImageToken = request.BestImageToken,
    method = request.Method
}
: new
{
    token1 = request.Token1,
    bestImageToken = request.BestImageToken,
    method = request.Method,
    tracking = new
    {
        extraData = request.TrackingToken,
        operationId = request.OperationId
    }
};
```

5. Lo que falta: regla de decisión y mapeo a HTTP

Este es el punto más importante que queda abierto. Hoy el handler devuelve el resultado tal cual y el endpoint responde `Results.Ok(result)` siempre: un rechazo biométrico llega al consumidor como HTTP 200.

5.1 La regla

```
public enum ValidationOutcome
{
    Approved,    // pasa: continuar con la activación del Soft Token
    Rejected,    // no es la misma persona, o no hay prueba de vida
    Retry,       // la captura no sirvió: pedir al cliente que repita
    Error        // falló Facephi o nuestra integración
}

private static ValidationOutcome Evaluate(PassiveLivenessResult r)
{
    // serviceResultCode == 0 sólo indica que el módulo se ejecutó.
    if (r.ServiceResultCode != 0)
        return ValidationOutcome.Error;

    var liveness = (int)r.passiveLivenessResult;

    // Errores técnicos del motor: no es culpa de la captura del cliente.
    if (liveness is 10 or 15)
        return ValidationOutcome.Error;

    // Rechazos reales: no coincide el rostro, o no se detectó vida.
    if (r.facialAuthenticationResult == 1 || liveness is 17 or 1)
        return ValidationOutcome.Rejected;

    // Lista blanca: sólo se aprueba con POSITIVE + Live explícitos.
    if (r.facialAuthenticationResult == 3 && liveness == 3)
        return ValidationOutcome.Approved;

    // Todo lo demás: la captura no permitió evaluar.
    return ValidationOutcome.Retry;
}
```

Lista blanca, nunca lista negra. No usar `liveness != 17` para decidir que hay vida: la documentación de `evaluatePassiveLiveness` muestra un rechazo NoLive que devuelve `0`, no `17`. Sólo el `3` aprueba.

5.2 Mapeo a HTTP

Outcome	HTTP	Qué hace el consumidor (APP-5223)
Approved	200	Continúa con la activación del Soft Token
Rejected	422	Corta el flujo. No reintentar la captura
Retry	422 con outcome: RETRY	Pide al cliente repetir la captura
Error	502	Falla técnica. Reintentable a nivel de servicio
Facephi 504	504	Timeout
Facephi 400 / 401 / 403	502 + log Error	El consumidor no puede corregirlo: es configuración nuestra

5.3 Respuesta sugerida del endpoint

Envolver el resultado sin perder los campos crudos, para que el consumidor no tenga que reimplementar la tabla:

```
public class ValidateIdentityV2Response
{
    /// <summary>APPROVED | REJECTED | RETRY | ERROR</summary>
    public string Outcome { get; set; } = string.Empty;

    public bool Approved { get; set; }

    /// <summary>Códigos de Facephi, para trazabilidad y soporte.</summary>
    public int FacialAuthenticationResult { get; set; }
    public int PassiveLivenessResult { get; set; }
    public double FacialAuthenticationSimilarity { get; set; }
    public string ServiceTransactionId { get; set; } = string.Empty;
}
```

Y en el endpoint:

```
public async Task<IResult> ValidateIdentityV2(
    ValidateIdentityV2Cmd request,
    ISender sender,
    CancellationToken cancellationToken)
{
    var result = await sender.Send(request, cancellationToken);

    return result.Outcome switch
    {
        "APPROVED" => Results.Ok(result),
        "ERROR"    => Results.Json(result, statusCode: StatusCodes.Status502BadGateway),
        _          => Results.Json(result, statusCode: StatusCodes.Status422UnprocessableEntity)
    };
}
```

Alternativa mínima si no se quiere cambiar la forma del body: dejar `PassiveLivenessResult` como respuesta y cambiar **sólo el código HTTP**. Es menos claro para el consumidor, pero no rompe nada de lo ya construido. Lo que **no** es aceptable es seguir devolviendo `200` para todo.

6. Lo que falta: headers, binding y limpieza

6.1 Headers obligatorios

Confirmar que `FacePhiService` envía ambos en el `HttpRequestMessage` :

```
httpRequest.Headers.Add("x-api-key", _appSettings.FACEPHI_IDENTITY_API_KEY);  
httpRequest.Headers.Add("family", "OnBoarding"); // requerido al enviar tracking
```

Como el payload sí incluye `tracking`, el header `family` aplica. Probar la capitalización que acepta el ambiente: la doc en español dice `OnBoarding` y la inglesa `Onboarding`.

6.2 Binding de la respuesta

`PassiveLivenessResult` mezcla convenciones: cinco campos en camelCase y cuatro en PascalCase. Los nombres son **correctos** para Identity V2, pero los cuatro en PascalCase sólo bindean si la deserialización es *case-insensitive*. Hacerlo explícito elimina la dependencia de esa configuración:

```
public class PassiveLivenessResult
{
    [JsonPropertyName("serviceTransactionId")]
    public string ServiceTransactionId { get; set; } = string.Empty;

    [JsonPropertyName("serviceResultCode")]
    public int ServiceResultCode { get; set; }

    [JsonPropertyName("serviceResultLog")]
    public string ServiceResultLog { get; set; } = string.Empty;

    [JsonPropertyName("serviceTime")]
    public string ServiceTime { get; set; } = string.Empty;

    [JsonPropertyName("facialAuthenticationResult")]
    public int facialAuthenticationResult { get; set; }

    [JsonPropertyName("facialAuthenticationLog")]
    public string facialAuthenticationLog { get; set; } = string.Empty;

    [JsonPropertyName("facialAuthenticationSimilarity")]
    public double facialAuthenticationSimilarity { get; set; }

    [JsonPropertyName("passiveLivenessResult")]
    public FacephiLivenessResult passiveLivenessResult { get; set; }

    [JsonPropertyName("passiveLivenessLog")]
    public string passiveLivenessLog { get; set; } = string.Empty;
}
```

Y un test que deserialice el JSON de ejemplo de la documentación, no un objeto construido en memoria:

```

[Test]
public void Deserializa_La_Respuesta_De_Ejemplo_De_Facephi()
{
    const string json = """
    {
        "serviceTransactionId": "2db602ee-3564-4304-af95-92a52eaae12d",
        "serviceResultCode": 0,
        "serviceResultLog": "[identity] Service executed ok",
        "serviceTime": "2235",
        "facialAuthenticationResult": 3,
        "facialAuthenticationLog": "Positive",
        "facialAuthenticationSimilarity": 0.99214232,
        "passiveLivenessResult": 3,
        "passiveLivenessLog": "Live"
    }
    """;

    var result = JsonSerializer.Deserialize<PassiveLivenessResult>(json)!;

    Assert.Multiple(() =>
    {
        Assert.That(result.ServiceResultCode, Is.EqualTo(0));
        Assert.That(result.ServiceTransactionId, Is.Not.Empty);    // detecta binding roto
        Assert.That(result.facialAuthenticationResult, Is.EqualTo(3));
        Assert.That((int)result.passiveLivenessResult, Is.EqualTo(3));
    });
}

```

6.3 Riesgo heredado: modelo compartido entre dos endpoints

`PassiveLivenessResult` se usa también en el flujo de onboarding que llama a `evaluatePassiveLiveness`. Ese endpoint devuelve `serviceLivenessResult`, no `passiveLivenessResult`. Si ambos flujos deserializan en el mismo modelo, el resultado de vida del flujo viejo estaría cayendo en `0` (None) de forma silenciosa.

Revisarlo. Si se confirma, separar en dos modelos: `IdentityValidationV2Result` y `PassiveLivenessResult`.

6.4 Nombres

Actual	Sugerido	Motivo
<code>EvaluatePassiveLivenessToken</code>	<code>ValidateIdentityV2Async</code>	Llama a Identity V2, no a <code>evaluatePassiveLiveness</code> , que es un endpoint distinto. Además falta el sufijo <code>Async</code> que usa el resto del servicio
<code>PassiveLivenessRequest</code>	<code>IdentityValidationV2Request</code>	Mismo motivo
<code>tests/.../SoftTokenFacephi/</code>	<code>tests/.../Facephi/</code>	Es justo el nombre que el equipo pidió no usar; el namespace dentro ya dice <code>Application.Facephi</code>

6.5 Manejo de errores

Revisar el `try/catch` de `FacePhiService`: si captura la excepción y devuelve un `PassiveLivenessResult` vacío, el resultado tendría `ServiceResultCode = 0` y la regla de la sección 5 lo leería como ejecución correcta. El catch debe propagar una excepción propia (por ejemplo `FacephiIntegrationException`) que el pipeline traduzca a `502` o `504`, nunca devolver un objeto vacío.

7. Checklist de cierre

#	Tarea	Sección	Prioridad
1	Implementar la regla de decisión <code>Approved/Rejected/Retry/Error</code>	5.1	Alta
2	Mapear el resultado a 200 / 422 / 502 / 504	5.2 – 5.3	Alta
3	Confirmar <code>x-api-key</code> y <code>family</code> en el request a Facephi	6.1	Alta
4	Revisar el <code>catch</code> : no devolver resultados vacíos	6.5	Alta
5	<code>[JsonPropertyName]</code> explícito + test de deserialización	6.2	Media
6	Verificar el modelo compartido con <code>evaluatePassiveLiveness</code>	6.3	Media
7	Renombrar servicio, request y carpeta de tests	6.4	Baja
8	Tests negativos: <code>method</code> inválido, <code>operationId</code> no-UUID, Facephi caído	—	Media
9	Corregir el modelo de datos de APP-5258: <code>token2</code> no aplica	1.1	Baja

7.1 Pruebas manuales

Aprobado

```
curl -sX POST https://localhost:5001/facephi/validate \  
  -H "Content-Type: application/json" \  
  -d '{"token1":"<base64-cedula>","bestImageToken":"<token-selfie>","method":"3"}'  
# → 200 { "outcome": "APPROVED", "approved": true, ... }
```

Con tracking

```
curl -sX POST https://localhost:5001/facephi/validate \  
  -H "Content-Type: application/json" \  
  -d '{"token1":"...","bestImageToken":"...","method":"5",  
      "tracking":{"extraData":"...","operationId":"123e4567-e89b-12d3-a456-426614174000"}}'
```

method inválido → 400 del validador

```
curl -sX POST https://localhost:5001/facephi/validate \  
  -H "Content-Type: application/json" \  
  -d '{"token1":"x","bestImageToken":"y","method":"4"}'
```

Verificar además que **ningún log** contenga `token1`, `bestImageToken` ni `extraData`.

8. Preguntas cerradas y abiertas

8.1 Cerradas

Pregunta	Respuesta
¿Endpoints propios de Soft Token?	No. Genéricos de Facephi
¿Qué endpoint se integra?	POST /onboarding/v2/identity
¿Importa el <code>requestId</code> ?	No
¿ <code>token2</code> (dorso)?	No aplica a Identity V2
¿Valores de <code>method</code> ?	"3" imagen abierta, "5" TokenFacelImage del widget SelphID
¿Identity V2 o Authenticate User V2?	Identity V2: el cliente escanea su cédula al activar el token
¿Persistencia en MongoDB?	No se requiere: la operación es sin estado

8.2 Abiertas

#	Pregunta	Para quién
1	¿Qué <code>method</code> usará la app: "3" con imagen abierta o "5" con widget SelphID? Define de dónde sale <code>token1</code>	Producto / Móvil
2	¿El endpoint <code>/facephi/validate</code> lleva autenticación, o queda interno tras el gateway?	Arquitectura / Seguridad
3	¿Cuántos reintentos se permiten ante <code>RETRY</code> antes de bloquear la activación?	Producto / Riesgo
4	¿Qué debe hacer APP-5223 ante <code>REJECTED</code> : bloquear, alertar a fraude, o ambas?	Producto / Riesgo
5	Capitalización correcta del header <code>family</code> : <code>OnBoarding</code> u <code>Onboarding</code>	Facephi

Documento de la implementación real de APP-5258, contrastado campo por campo contra la documentación oficial de Facephi Identity API. Los fragmentos de código de las secciones 5 y 6 son propuestas de cambio y no han sido compilados.